

## Software-Entwicklung mit JavaScript

# Übungsblatt 4

## DOM – DOM-Ereignisse

27. 11. 2025

Wintersemester 2025 – 2026

### Übung 1. *Kurze Wiederholung*

Lesen Sie die Dokumentation über die Methoden

- `document.getElementById`,
- `Element.innerHTML`, `Element.addEventListener`
- `HTMLDataElement.value`
- so wie über die Konstruktion eines HTML `table`-Tag

aus <https://developer.mozilla.org/en-US/>. Schreiben Sie das Programm Calendar aus der Übung 1 um, sodass der Kalender aus einem `table`-Tag besteht und an einem vordefinierten HTML-Element im DOM angehängt wird. Ferner soll der Benutzer den Monat und das Jahr in 2 unterschiedlichen `input` eingeben und auf einem `button` anklicken können, um der Kalender anzeigen zu lassen.

Sie benutzen das Objekt `terminal` so wie die Stylesheet `dfhi.css` und `dom-output.css` in dieser Übung nicht. Gestalten Sie den Kalender so, wie er Ihnen am besten gefällt.

### Übung 2. *Zeitaufwand der Vorlesung*

Der gesamte Aufwand einer Vorlesung ist durch die ECTS-Zahl zu erkennen. Ein ECTS entspricht 30 Zeitstunden (jeweils 60 Minuten). Eine Doppelstunde (Vorlesung, Tutorial, Übung, ect.) entspricht zwei Unterrichtsstunden (jeweils 45 Minuten). Die Vorlesungszeit beträgt in der Regel 15 Woche.

1. Berechnen Sie den Zeitaufwand für die Vor- und Nacharbeit der Vorlesung „Softwareentwicklung mit JavaScript“ in eine Woche. Die Vorlesung hat fünf ECTS. In der Vorlesungszeit haben Sie drei (mit freiwilligen Übungsstunden) oder zwei (ohne freiwillige Übungsstunden) Doppelstunden (DS). Sie müssen diese zwei Varianten berechnen. Die Zeitaufwand wird in Zeitstunden gerechnet.
2. Erstellen Sie ein JavaScript Programm, das den Zeitaufwand pro Woche, die ein Student erbringen muss, in einem Semester.
  - a) Das Programm besteht aus die Datei `lecture.html` und `lecture.js`. Sie verwenden in diesem Programm den Stylesheet `dfhi.css` so wie `dom-output.css` nicht.
  - b) Erstellen Sie eine JavaScript-Funktion `lectureEffortPerWeek`, die den Zeitaufwand für die Vor- und Nacharbeit einer Vorlesung in eine Woche berechnet. Die Funktion nimmt der Anzahl der ECTS, der Vorlesungszeit (in Woche) und der Anzahl der DS in der Woche als Argumenten entgegen und gibt die Zeitaufwand pro Woche in Stunden zurück.
  - c) Erstellen Sie in der HTML-Datei die `<input>`-Tags für jeweiligen Informationen, die für die Berechnung des Aufwands gebraucht wird. Ihre Benutzer werden diese Elementen ausfüllen, erstellen Sie deshalb auch die entsprechenden Labels für jeweiligen `<input>`-Tags.
  - d) Erstellen Sie einen Button mit dem Id `calculateEffort`.
  - e) Erstellen Sie eine JavaScript-Funktion `calculateEffort`, die die Werten der `<input>`-Tags abliest, in entsprechend Datentypen konvertiert und die Funktionen `lectureEffortPerWeek` aufruft um den Aufwand zu berechnen.

Das Ergebnis soll in einem `<span>`-Tag mit dem Id `lectureEffort` hinzugefügt werden.
  - f) Verknüpfen Sie die Funktion `calculateEffort` mit dem Ereignis „click“ vom Button „calculate“.

### Übung 3. Zeitaufwand eines Semester

Ein Semester besteht aus mehr als eine Vorlesung. Erweitern Sie das Programm `lecture.html|js` so, dass der Benutzer auch Information mehrere Vorlesung erfassen kann. Das Programm soll die Informationen der Vorlesungen in eine Tabelle darstellen und den gesamten Zeitaufwand für die Vor- und Nacharbeit aller Vorlesungen zusammen rechnen und darstellen.

Entwerfen Sie zuerst einen Prototyp Ihres Programm mit Stift und Papier. Erstellen Sie dann die CSS-Regel in einer Datei `lecture.css` und binden Sie diese Datei in der HTML-Datei ein. Benutzen Sie die `flex`-Features um Ihre Layout zu gestalten.

### Übung 4. Generator

Untersuchen Sie das `Generator`-Objekt aus MDN-Seite

[https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\\_Objects/Generator](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator).

Implementieren Sie die Aufgabe 7 in *Check Your Skill №0* mit Generator. Der Generator-Constructor `countPropagate` soll folgende Signatur wie im untenstehenden Listing haben.

Der Aufrufer soll die Anzahl der untersuchenden Minuten mit dem Argument `m` festlegen können. Die Ausgabe der Verbreitung des Virus „Crash||Cash“ soll nicht in der Constructor-Funktion `countPropagate` implementiert werden.

Benutzen Sie in diesem Programm wie übrig das Objekt `console`. Wichtige Punkten sind:

1. Die Constructor-Funktion `countPropagate` hat keine explizit Exponent-Operation (Keine Nutzung von `Math.pow` oder den Operator `**`)
2. Die Ergebnisse der vorherigen Berechnung werden wieder verwendet, um die Anzahl der Operationen zu minimieren.

```
function* countPropagate(n, k, m=Number.MAX_SAFE_INTEGER) {
    // ....
}

const crashOrCash = countPropagate(5, 1.6);

for( const infectedState of crashOrCash ) {
    console.log(infectedState);
}

// Ausgabe in der Console könnte (schematisch) wie folge aussehen:
/*
[0, 5, 5]
[1, 8, 8]
[2, 13, 12.8]
[3, 20, 20.480000000000004]
[4, 33, 32.768000000000001]
[5, 52, 52.428800000000002]
[6, 84, 83.886080000000004]
[7, 134, 134.217728000000005]
[8, 215, 214.74836480000001]
[9, 344, 343.59738368000002]
[10, 550, 549.75581388800003]
*/
```

### Übung 5.

Erweitern Sie die Aufgabe 4 so, dass der Benutzer die Anzahl der infizierten Geräten  $n$  und die Verbreitung-Rate eingeben kann. Der Benutzer kann die Verbreitung in den ersten 10 Minuten sehen, dann kann er die Verbreitung in den nächsten Minuten mit einem Anklickt auf dem Button „Next“ anzeigen lassen.

Wenden Sie die Vorteil von Lazy-Evaluierung der Generator hier an.